CSC1024
PROGRAMMING PRINCIPLES

FINAL PROJECT:
SOCIAL MEDIA CONTENT PLANNER

Practical 8

Team: Muhammad Abbas (26052019), Eeh Cheng Zher (25023896), Chin Keen Yen (25027962) and Gam Yao Teng (25037318)

THE TEAM

Muhammad Abbas
26052019
Project Dictator

Eeh Cheng Zher
25023896
Team Member

Chin Keen Yen
25027962
Team Member

Gam Yao Teng
25037318
Team Member

PART 1:
MAIN MENU & POST CREATION

Presenter : Eeh Cheng Zher

MAIN MENU:

- DISPLAY SELECTION FOR USER TO CHOOSE
- CALL FOR APPROPRIATE FUNCTIONS BASED ON INPUT

POST CREATION:

- ALLOWS USER TO ADD NEW POST
- AUTOMATICALLY SAVES TO FILE
- FIELDS: ID, PLATFORM, CAPTION, DATE, STATUS

SHOWCASE

MAIN MENU

EXPLANATION

MAIN MENU

```python
def main_menu():
    while True:
        heading("Social Media Content Planner")
        print("1. Add New Post")
        print("2. Update Post Status")
        print("3. Record Engagement Metrics")
        print("4. Display Content Calendar")
        print("5. Generate Performance Report")
        print("6. Export Report to File")
        print("7. Exit")
        initial_choice = input("\nEnter your choice: ").strip()
        match initial_choice: #match-case is neat compared to long if/elif chain
            case "1":
                new_post_menu()
            case "2":
                post_status_menu()
            case "3":
                engagement_entry()
                time.sleep(2)
            case "4":
                content_calendar_menu()
            case "5":
                generate_report_menu()
            case "6":
                export_report()
                time.sleep(2)
            case "7":
                print("Exiting Program...")
                break #exits the while True loop, ending the program
            case _:
                print("Enter valid choice (1-7)")
```

SHOWCASE

POST CREATION

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice:
```

EXPLANATION

POST CREATION

```python
def new_post_menu():
    while True:
        subheading("Add New Post")
        print("1. Draft New Post")
        print("2. Go Back")
        first_choice = input("\nEnter your choice: ").strip()
        match first_choice:
            case "1":
                draft_new_post()
            case "2":
                return #return back to main_menu() loop
            case _:
                print("Enter valid choice (1/2)")
```

```python
def draft_new_post():
    # Abbas validation file, removed try catch block previously in its place
    file_check("posts.txt")
    file_check("platforms.txt")
    while True:
        post_id = validate_required_input("\nEnter Post ID: ")
        if validate_unique_id(post_id):
            break
        # end of abbas validation file

    print("\nAvailable Platforms")
    platform_list = []
    try:
        with open("platforms.txt","r") as file:
            for line in file:
                platform = line.strip().split(",")
                platform_list.append(platform[1]) #platform[1] = platform name
                print(f"- {platform[1]}")
    except FileNotFoundError:
        print("platforms.txt not found")
        return #cannot validate platform choice without file, so stop

    #Abbas validation file
    while True:
        platform = input("\nEnter Platform: ").strip()
        if validate_platform(platform): # validates platform using platforms.txt
            break
        print("Invalid platform. Please choose from the list above.")

    caption = validate_required_input("Enter Caption: ") # ensures non-empty caption
    schedule_date = validate_date("Enter Scheduled Date (DD/MM/YYYY): ") # validates date format
    #end of validation file

    status = "Draft" #every new created post starts with Draft status
    with open("posts.txt","a") as file: #"a" = append mode, add lines without erasing existing posts
        file.write(f"{post_id},{platform},{caption},{schedule_date},{status}\n") #comma separated to match read/split format in file
    notification("Post Added Successfully!")
    print(f"---> [Saved into posts.txt]")
    print(f"    ID: {post_id} | Platform: {platform} | Caption: {caption} | Schedule: {schedule_date} | Status: {status}") #display field separation
using "|"
    time.sleep(2)
```

posts.txt

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Draft

PART 2:
POST MANAGEMENT & CONTENT CALENDAR

Presenter : Chin Keen Yen

POST MANAGEMENT:

- ALLOWS USER TO UPDATE POSTS STATUS
- DYNAMICALLY ADAPTS OPTIONS BASED ON CURRENT STATUS

CONTENT CALENDAR:

- DISPLAYS SUMMARY OF ALL POSTS
- INTELLIGENTLY SORT POSTS BY DATE

SHOWCASE

POST MANAGEMENT

DRAFTED
\/
SCHEDULED
\/
POSTED

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice:
```

EXPLANATION

POST MANAGEMENT

```python
def post_status_menu():
    while True:
        subheading("Update Post Status")
        print("1. Update Status")
        print("2. Go back")
        choice = input("\nEnter your choice: ").strip()
        match choice:
            case '1':
                update_post_status()
                time.sleep(2)
            case '2':
                return #returns back to main menu loop
            case _:
                print("Enter a valid option (1-2)")
```

```
def update_post_status():
    notification("Update Post Status")

    post_id = validate_required_input("Enter the Post ID to update: ") #now goes to validation.py

    try:
        with open("posts.txt", "r") as file: #opens and reads posts.txt
            lines = file.readlines() #reads posts.txt file line by line
    except FileNotFoundError: #catch cases where the posts.txt file does not exist
        print("No posts to be updated, create a post and try again.")
        return #stops after post ID cannot be found

    post_found = False #checks if desired post was found, initialise status as not found before loop
begins
    new_lines = [] #list to hold lines to be written back into posts.txt

    for line in lines: #goes through each line already saved in posts.txt
        data = line.strip().split(",")
        if data[0].upper() == post_id.upper(): #designates data[0] as post ID
            post_found = True #check if post ID was found, status gets updated to true
            current_status = data[4] #stores the post current status and is used to check for future updates
to status
            print(f'\nPost ID: {data[0]}')
            print(f'Platform: {data[1]}')
            print(f'Caption: {data[2]}')
            print(f'Scheduled Date: {data[3]}')
            print(f'Current Status: {current_status}')
```

posts.txt

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Draft

```
 if current_status == "Draft": #allows posts with 'draft' status to update to scheduled or posted
          print("\n1. Update to Scheduled")
          print("2. Update to Posted")
          print("3. Cancel")
          status_choice = input("Enter option (1-3)\n")

          if status_choice == '1':
             data[4] = "Scheduled" #updates the current status into scheduled from draft
             print("Post Status has been updated to Scheduled.")
          elif status_choice == '2':
             data[4] = "Posted" #updates current status from draft to posted
             print("Post Status has been updated to Posted.")
          else:
             print("Status update cancelled.")

       elif current_status == "Scheduled":
          confirm_status = input("\nUpdate status to Posted? (y/n): ").strip() #scheduled posts can only be updated to posted and not back to draft
          if confirm_status == 'y':
             data[4] = "Posted"
             print("Status has been updated to Posted.")
          else:
             print("Status update cancelled.")
       elif current_status == "Posted": #already posted, no further changes can be done
          print("\nThis post has already been posted, no updates can be made.")
       else:
          print("\nPost Status Unknown... Unable to update.")

       new_line = ",".join(data) #separates the new lines with commas
       new_lines.append(new_line + "\n")
    else:
       new_lines.append(line) #adds back lines that dont match post ID back without changes

if not post_found: #could not find a post ID to match user input
    print("Post ID not found.")
    return #exits since there is nothing to update
with open("posts.txt", "w") as file: #opens posts.txt in write mode
    file.writelines(new_lines) #rewrites entire updated list into posts.txt
```

Option 1

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Draft

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Scheduled

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Posted

Option 2

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Draft

1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Posted

SHOWCASE

CONTENT CALENDAR

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice:
```

EXPLANATION

CONTENT CALENDAR

```
def content_calendar_menu():
    while True:
        display_content_calendar()
        time.sleep(2)      # adds a delay of 2 seconds, before the Main Menu pushes the table higher up
        return      # exits loop after 2 second delay

def sorted_calendar(data):
    return data[3].split("/")[::-1]
    #reverses the order of data[3] which is the scheduled date column, into YYYY/MM/DD in order to sort by year first instead of day, does not affect the final display
    #split("/") separates the day, month, year into respective fields, [::-1] reverses the order into year, month, day
```

```
def display_content_calendar():
    notification("Content Calendar")
    try:
        with open("posts.txt", "r") as file:
            lines = file.readlines() #reads all lines in posts.txt
    except FileNotFoundError: #for cases where posts have not been created yet
        print("Post does not exist.")
        return
    if len(lines) == 0: #for cases where posts have been created but are empty
        print("Post does not exist.")
        return

    posts = [] #new list to temporarily store the sorted schedule dates
    for line in lines:
        posts.append(line.strip().split(",")) #adds each new line into the temporary posts list and splits with a comma

    posts.sort(key=sorted_calendar, reverse=True)
    #'key=' specifically calls on 'sorted_calendar' to sort the posts by their scheduled date oldest first, using the 'sorted_calendar' to compare
    #'reverse=True' because without it the scheduled dates are ordered with the older posts sorted to the top

    #for the column headers, formatted to ensure the table is aligned and fix the width
    print(f"{'Post ID':<10}{'Platform':<15}{'Caption':<25}{'Date':<14}{'Status':<10}")
    print("-" * 74) #prints - to match the width of the all column header width
```

```
for data in posts: #loops through the prior posts list and assigns each data index to a column to match
        post_id = data[0]
        platform = data[1]
        caption = data[2]
        schedule_date = data[3]
        status = data[4]

        if len(caption) > 20: #prevent text from overflowing to next column after 20 letters
            caption = caption[0:20] + "..." #adds '...' after the 20 letters

        #left aligning each column field to each respective field width
        print(f"{post_id:<10}{platform:<15}{caption:<25}{schedule_date:<14}{status:<10}")
```

PART 3:
ENGAGEMENT TRACKING & PERFORMANCE REPORT

Presenter : Gam Yao Teng

ENGAGEMENT TRACKING:

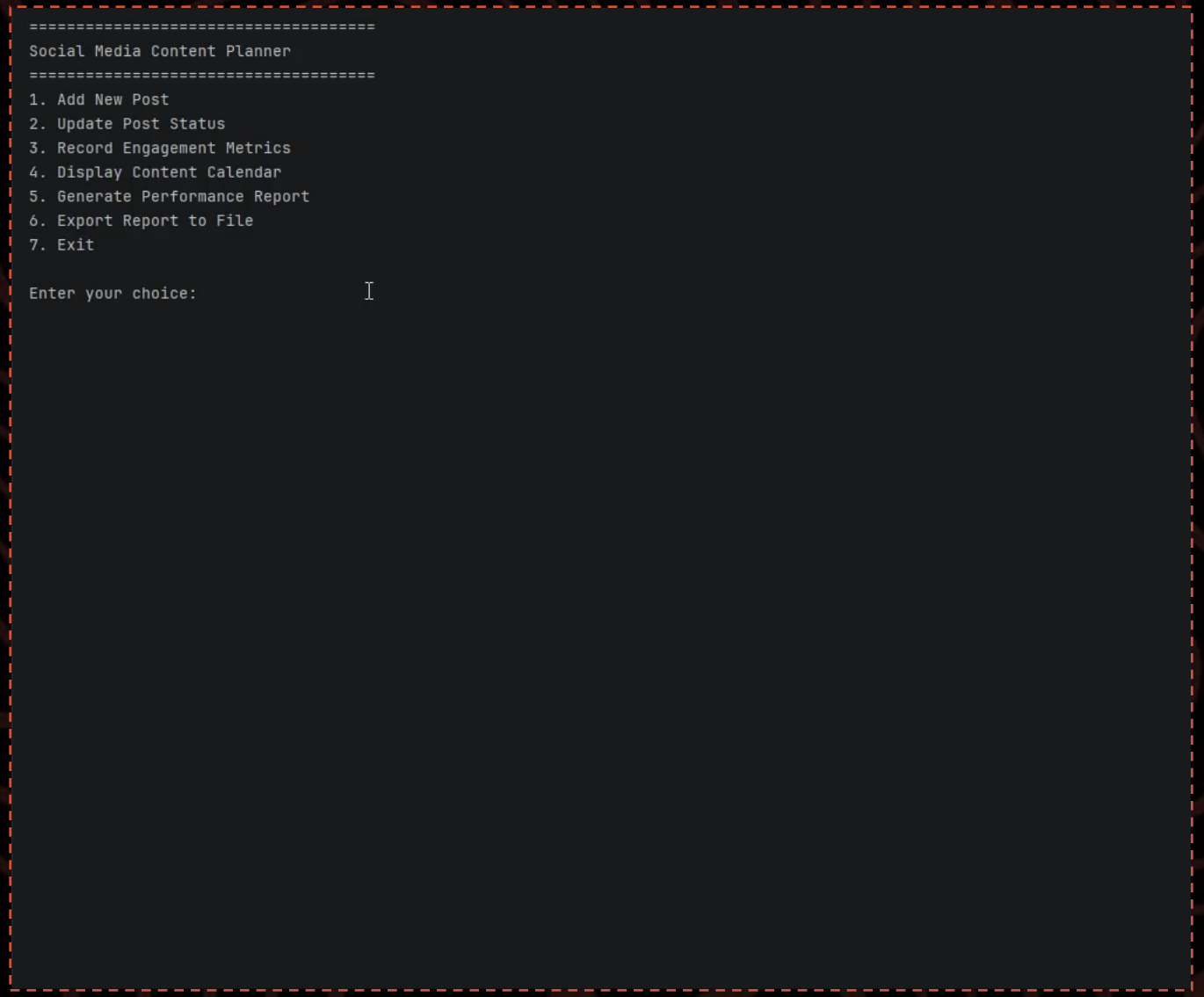
- ALLOWS USER TO RECORD ENGAGEMENT METRICS
- SMARTLY RESTRICTS ENGAGEMENT RECORD TO “POSTED” POSTS

PERFORMANCE REPORT:

- GENERATES VARIOUS PERFORMANCE REPORT
- ALGORITHM-BASED PERFORMANCE CALCULATION
- CAPABLE OF EXPORTING REPORT

SHOWCASE

ENGAGEMENT TRACKING



EXPLANATION

ENGAGEMENT TRACKING

```
1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Posted
8,Facebook,Flash sale,24/03/2026,Scheduled
9,X,Did you know?,28/07/2026,Draft
10,Facebook,Industry news,12/08/2026,Draft
11,Facebook,Company update,29/08/2026,Scheduled
12,Facebook,Giveaway alert,07/01/2026,Posted
```

```
def engagement_entry():    #allows user to record engagement metrics
    file_check("engagement.txt")    #runs the function to check whether engagement.txt
exists
    subheading("Record Engagement Metrics")
    valid_posts = posted_check()    #gets a list of posts with "Posted" status
    print("Please choose from one of these posts to log engagement data.\n")
    for i in valid_posts:      #prints out every valid posts
        print(i)

print("==========================================================================
=")
```

```
def posted_check():
    #checks and creates a list of post IDs that have the "Posted" status
    posted_posts= []
    with open("posts.txt", "r") as file:
        for row in file:
            data = row.strip().split(",")   #splits each row based on separator ","
            if data[4] == "Posted":        #reads the 4th index, which is the "Status" column
                posted_posts.append(data[:4])   #if is "Posted", appends to list while omitting the
status itself
            else:
                pass
    return posted_posts     #returns the list
```

```
while True:
    inputID = input("Enter your post ID [Q to quit]: ")
    if inputID.strip().upper() != "Q":      #if user input is not "Q", proceed with next line, otherwise break from this function
        if not id_verification(inputID):    #if inputted ID is already inside engagement.txt, tells user to enter another ID
            if posts_data_extraction(inputID) is not None:  #if the returned value is not None, or contains something, proceed with next line, otherwise tells user to enter a valid ID
                data = posts_data_extraction(inputID)       #gets the data of the inputted post ID
                post_ID = data[0]
                platform = data[1]
                print(f"\nYou are currently adding entry for {post_ID}.")
                #use int_validity(userin) to check whether user input is an integer without causing an error

                # now using the validation file to verify instead
                views = validate_positive_integer("Enter the number of views: ")
                likes = validate_positive_integer("Enter the number of likes: ")
                comments = validate_positive_integer("Enter the number of comments: ")
                shares = validate_positive_integer("Enter the number of shares: ")
                #end of abbas contribution

                with open(f"engagement.txt", "a") as file:      #opens engagement.txt and appends the data inputted inside
                    file.write(f"{post_ID},{platform},{views},{likes},{comments},{shares}\n")
                print(f"\n======================================"
                    f"\nEngagement data successfully logged.\n"
                    f"\nPostID, Platform, Views, Likes, Comments, Shares"
                    f"\n{post_ID}, {platform}, {views}, {likes}, {comments}, {shares}\n")
                break
            else:
                print("\n============================"
                    "\nPlease enter a valid post ID.")
        else:
            print("\n============================================================"
                "\nThis ID already has a record, please enter another post ID.")
    else:
        print("\n====================="
            "\nOperation cancelled.\n")
        break
```

```
def posts_data_extraction(userin):
    #returns the data for the entire row of post ID based on user specified ID
    valid_posts = posted_check()
    for row in valid_posts:
        if row[0] == userin.upper():
            return row      #returns the entire row of data
```

engagement.txt

```
7,Facebook,10976,909,240,42
```

```
7,Facebook,10976,909,240,42
12,Facebook,2324,501,279,119
```

SHOWCASE

GENERATE PERFORMANCE REPORT

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice:
```

EXPLANATION

GENERATE PERFORMANCE REPORT

```
def generate_report_menu():
    #generates a menu to print performance report
    while True:
        subheading("Performance Report")
        print("\n1. Total Posts"
              "\n2. Best Performing Post"
              "\n3. Highest Interacted Platform"
              "\n4. Print All"
              "\n5. Return")
        choice = input("\nEnter your choice: ")
        match choice.strip():
            case "1":
                print(total_posts())
                time.sleep(2)  # adds a delay of 2 seconds, before the Main Menu pushes the table
higher up
            case "2":
                print(best_performing())
                time.sleep(2)
            case "3":
                print(highest_interaction())
                time.sleep(2)
            case "4":
                print(total_posts())
                print(best_performing())
                print(highest_interaction())
                time.sleep(2)
            case "5":
                break
            case _:
                print("Enter a valid choice (1-5)")
```

```python
def total_posts():   #calculates the total posts from each platform
    fb = ig  =  tt  =  x  = 0
    data = posted_check()      #gets the data of all posts with "Posted" status
    for row in data:          #sort each row of data by platform
        platform = row[1]
        if platform.lower()== "facebook":
            fb += 1
        elif platform.lower()== "instagram":
            ig += 1
        elif platform.lower()== "tiktok":
            tt += 1
        elif platform.lower()== "x":
            x += 1
    total_posted = ig + fb + ig + tt + x
    return(f"\n\nTotal Posts From All Platforms"
           f"\n============================"
           f"\nFacebook -- {fb}"
           f"\nInstagram -- {ig}"
           f"\nTikTok -- {tt}"
           f"\nX -- {x}"
           f"\nTotal Posted -- {total_posted}"
           f"\n============================")
```

```python
def posted_check():
    #checks and creates a list of post IDs that have the "Posted" status
    posted_posts= []
    with open("posts.txt", "r") as file:
        for row in file:
            data = row.strip().split(",")   #splits each row based on separator ","
            if data[4] == "Posted":        #reads the 4th index, which is the "Status"
column
                posted_posts.append(data[:4])   #if is "Posted", appends to list while
omitting the status itself
            else:
                pass
    return posted_posts     #returns the list
```

```
1,TikTok,Industry news,05/01/2026,Draft
2,TikTok,Company update,22/07/2026,Scheduled
3,Instagram,New product launch!,05/10/2026,Draft
4,Facebook,Industry news,23/05/2026,Draft
5,Facebook,Flash sale,27/03/2026,Scheduled
6,X,Customer testimonial,18/06/2026,Draft
7,Facebook,Customer testimonial,02/09/2026,Posted
8,Facebook,Flash sale,24/03/2026,Scheduled
9,X,Did you know?,28/07/2026,Draft
10,Facebook,Industry news,12/08/2026,Draft
11,Facebook,Company update,29/08/2026,Scheduled
12,Facebook,Giveaway alert,07/01/2026,Posted
```

```
def best_performing():      #calculate and find the best performance post
    try:
        best_score = 0
        best_post = None        #sets the current best post as none to prevent error
        with open("engagement.txt", "r") as file:       #try to open engagement.txt in read mode, if file does not exists, prompt user to
enter engagement metrics
            for row in file:        #go through each row in engagement.txt
                data = row.strip().split(",")
                post_id = data[0]
                view_score = int(data[2]) * 0.004       #based on the number of views, likes, comments, and shares, calculate the
performance score
                like_score = int(data[3]) * 0.003
                comment_score = int(data[4]) * 0.001
                shares_score = int(data[5]) * 0.002
                performance_score = view_score + like_score + comment_score + shares_score
                if performance_score > best_score:      #compare current performance score with previous best score
                    best_score = performance_score      #if current score is greater than previous score,
                    best_post = post_id                 #sets the current post as best performing post
                    best_view = data[2]
                    best_like = data[3]
                    best_comment = data[4]
                    best_share = data[5]
        if best_post is None:   #if engagement.txt contains no data, prompt user to enter engagement metrics
            return("\n\n============================================================"
                   "\nNo engagement data found, please enter engagement metrics first.\n")
        return(f"\n\nBest Performing Post"
               f"\n============================"
               f"\nPost ID -- {best_post}"
               f"\nNo. Views -- {best_view}"
               f"\nNo. Likes -- {best_like}"
               f"\nNo. Comments -- {best_comment}"
               f"\nNo. Shares -- {best_share}"
               f"\nPerformance Score -- {best_score:.2f}"
               f"\n============================\n")
    except FileNotFoundError:
        return("\n============================================================"
               "\nengagement.txt does not exists, please enter engagement metrics first.\n")
```

engagement.txt

```
7,Facebook,10976,909,240,42
12,Facebook,2324,501,279,119
16,Instagram,909,969,68,98
21,Instagram,12649,1256,198,70
24,Facebook,5006,619,45,143
28,Instagram,4512,1707,287,57
31,Facebook,4157,1830,155,84
35,X,2786,63,190,15
36,Instagram,12566,1604,300,59
38,TikTok,2179,1700,103,9
40,Facebook,11587,376,40,81
42,TikTok,12635,1479,28,103
43,Facebook,9435,915,121,69
44,TikTok,1924,746,153,17
45,TikTok,10174,619,45,55
49,Instagram,7412,368,124,146
50,TikTok,1020,490,56,81
53,TikTok,988,1613,199,55
55,Facebook,2035,739,147,128
56,X,4082,259,237,102
57,Facebook,4311,239,191,118
58,Instagram,8779,828,88,37
62,TikTok,10363,248,194,68
```

```
def highest_interaction():   #finds the platform with the highest interaction
    try:
        best_score = 0
        best_platform = None
        fb_score = 0
        ig_score = 0
        tt_score = 0
        x_score = 0
        with open("engagement.txt", "r") as file:   #try to open engagement.txt in read mode, if file does not exists, prompt user to enter engagement metrics
            for row in file:                    #sorts each row in engagement.txt by platform
                data = row.strip().split(",")
                platform = data[1]
                match platform:                 #calculates interaction score based on each platform
                    case "Facebook":
                        fb_score += interaction_formula(int(data[2]),int(data[3]),int(data[4]),int(data[5]))
                    case "Instagram":
                        ig_score += interaction_formula(int(data[2]),int(data[3]),int(data[4]),int(data[5]))
                    case "TikTok":
                        tt_score += interaction_formula(int(data[2]),int(data[3]),int(data[4]),int(data[5]))
                    case "X":
                        x_score += interaction_formula(int(data[2]),int(data[3]),int(data[4]),int(data[5]))
```

```
def interaction_formula(view,like,comment,shares):
    #calculate interaction score
    view_score = view * 0.001
    like_score = like * 0.002
    comment_score = comment * 0.004
    shares_score = shares * 0.003
    interaction_score = view_score + like_score + comment_score + shares_score
    return interaction_score    #returns the interaction score
```

```
7,Facebook,10976,909,240,42
12,Facebook,2324,501,279,119
16,Instagram,909,969,68,98
21,Instagram,12649,1256,198,70
24,Facebook,5006,619,45,143
28,Instagram,4512,1707,287,57
31,Facebook,4157,1830,155,84
35,X,2786,63,190,15
36,Instagram,12566,1604,300,59
38,TikTok,2179,1700,103,9
40,Facebook,11587,376,40,81
42,TikTok,12635,1479,28,103
43,Facebook,9435,915,121,69
44,TikTok,1924,746,153,17
45,TikTok,10174,619,45,55
49,Instagram,7412,368,124,146
50,TikTok,1020,490,56,81
53,TikTok,988,1613,199,55
```

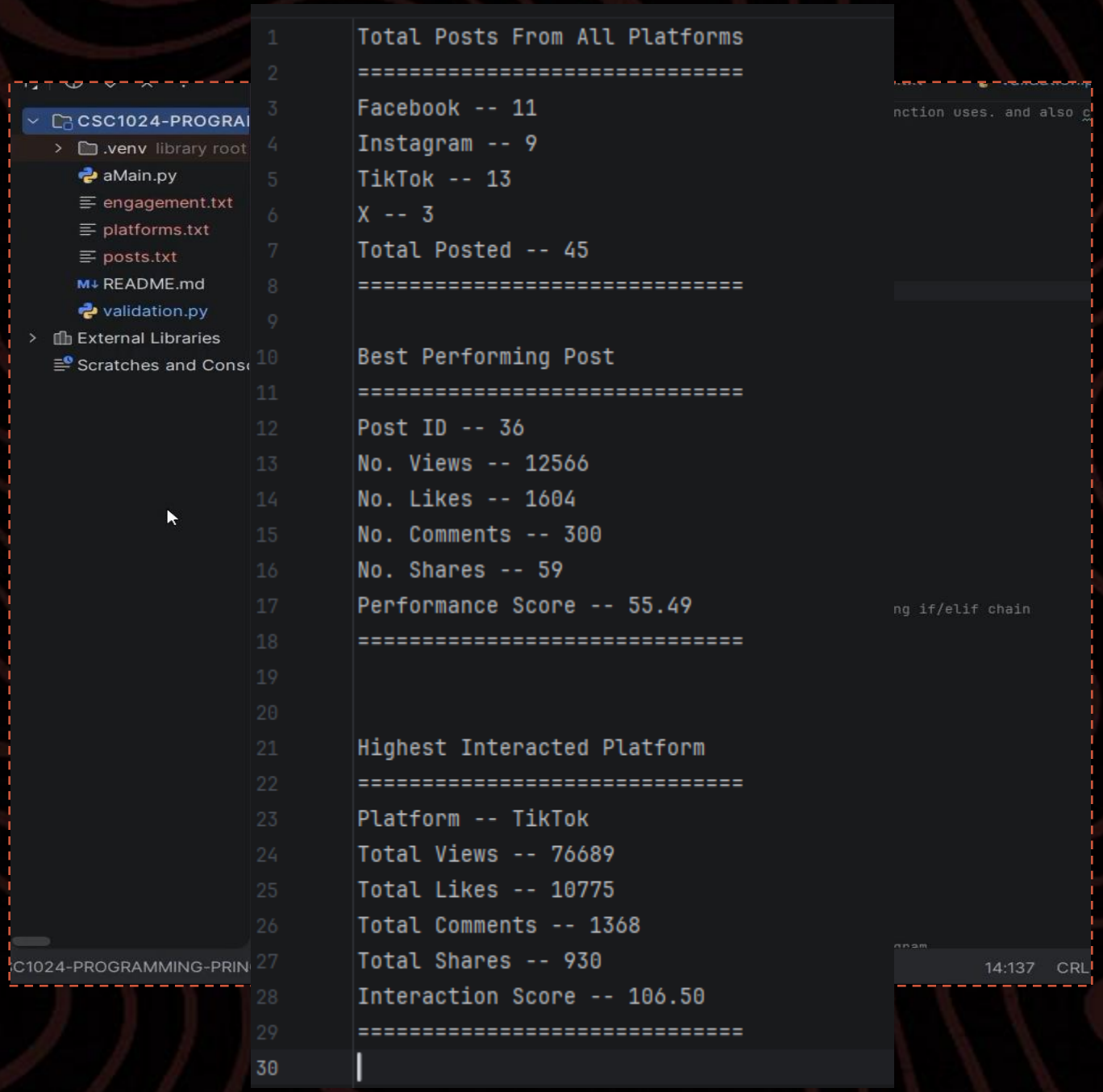
```
#compares score of each platform to determine the highest value
    best_score = max(ig_score, fb_score, tt_score, x_score)
    best_platforms = []

    if ig_score == best_score:  # makes it possible to identify multiple platforms with highest views and display all names
        best_platforms.append("Instagram")
    if fb_score == best_score:
        best_platforms.append("Facebook")
    if tt_score == best_score:
        best_platforms.append("TikTok")
    if x_score == best_score:
        best_platforms.append("X")

    best_platform = ", ".join(best_platforms)  #merges all values from best_platforms separating them with ", " converts from array to string var

    platform_interaction = total_interaction(best_platform)     #calculates the total interactions from said platform
    return(f"\n\nHighest Interacted Platform"
           f"\n============================="
           f"\nPlatform -- {best_platform}"
           f"\n{platform_interaction}"
           f"\nInteraction Score -- {best_score:.2f}"
           f"\n=============================\n")
except FileNotFoundError:

return("\n\n============================================================"
           "\nengagement.txt does not exists, please enter engagement metrics first.\n")
```

```
def total_interaction(platform):
    #calculate total interactions from the specified platform
    views = 0
    likes = 0
    comments = 0
    shares = 0
    with open("engagement.txt", "r") as file:
        for row in file:
            data = row.strip().split(",")
            if data[1] == platform:
                views += int(data[2])
                likes += int(data[3])
                comments += int(data[4])
                shares += int(data[5])
    return(f"Total Views -- {views}"
           f"\nTotal Likes -- {likes}"
           f"\nTotal Comments -- {comments}"
           f"\nTotal Shares -- {shares}")
```

```
7,Facebook,10976,909,240,42
12,Facebook,2324,501,279,119
16,Instagram,909,969,68,98
21,Instagram,12649,1256,198,70
35,X,2786,63,190,15
38,TikTok,2179,1700,103,9
42,TikTok,12635,1479,28,103
43,Facebook,9435,915,121,69
44,TikTok,1924,746,153,17
45,TikTok,10174,619,45,55
49,Instagram,7412,368,124,146
50,TikTok,1020,490,56,81
53,TikTok,988,1613,199,55
```

SHOWCASE

EXPORT PERFORMANC E REPORT



EXPLANATION

GENERATE PERFORMANCE REPORT

```
def export_report():
    #export performance report with user specified name
    try:
        with open("engagement.txt", "r") as file:      #try to open engagement.txt in read mode, if file does not exists, prompt user to enter engagement metrics
            pass
        total_posts_report = total_posts()              #print out and assign every mini report to their own unique variable
        best_performing_report = best_performing()
        highest_interaction_report = highest_interaction()
        subheading("Exporting Performance Report..")
        while True:
            report_name = input("\nEnter the name of the file (Don't include file extensions) [Q to quit]: ") #prompts user to enter name of report
            if report_name.strip() != "":      #checks if user entered nothing, if yes then prompt user to enter again
                if report_name.strip().upper()!= "Q":      #checks if user entered "Q", if yes then break from this function
                    with open(f"{report_name}.txt", "w") as file:      #open a new file with user specified name
                        for i in [total_posts_report.lstrip(), best_performing_report, highest_interaction_report]:
                            file.write(i)      #writes the mini reports into it
                    print(f"\n\nPerformance Report successfully exported as {report_name}.txt"
                          f"\n==========================================================\n")
                    break
                else:
                    print("\n===================="
                          "\nOperation cancelled.\n")
                    break
            else:
                print("Please enter a valid file name.")
    except FileNotFoundError:
        print("\n\n=========================================================="
              "\nengagement.txt does not exists, please enter engagement metrics first.\n")
```

PART 4:
INPUT VALIDATION AND ERROR CATCHING

Presenter : Muhammad Abbas

INPUT VALIDATION & ERROR CATCHING:

- EMPLOY VALIDATING FUNCTIONS TO CATCH AND PREVENT ERRORS
- PREVENT USER ERRORS BY FORCING INPUT
- ALLOWS PROGRAM TO FUNCTION AS INTENDED WITHOUT CRASHING

SHOWCASE

FILE NON-EXISTENCE PREVENTION

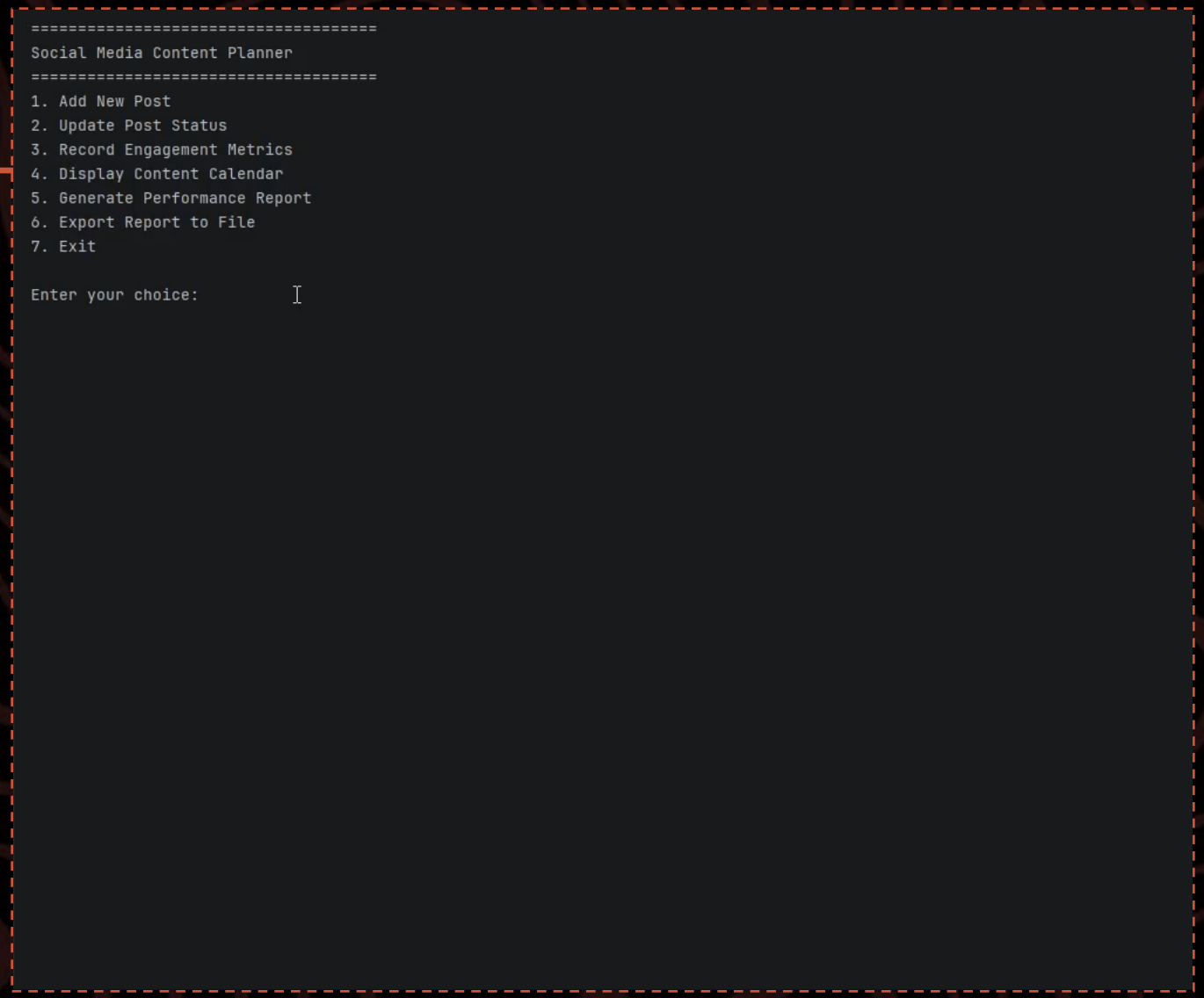
posts.txt
platforms.txt

```
====================================
Social Media Content Planner
====================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 1

--------------------------
Add New Post
----------------------
1. Draft New Post
2. Go Back

Enter your choice:
```

SHOWCASE

FILE NON-EXISTENCE PREVENTION

Engagement.txt



EXPLANATION

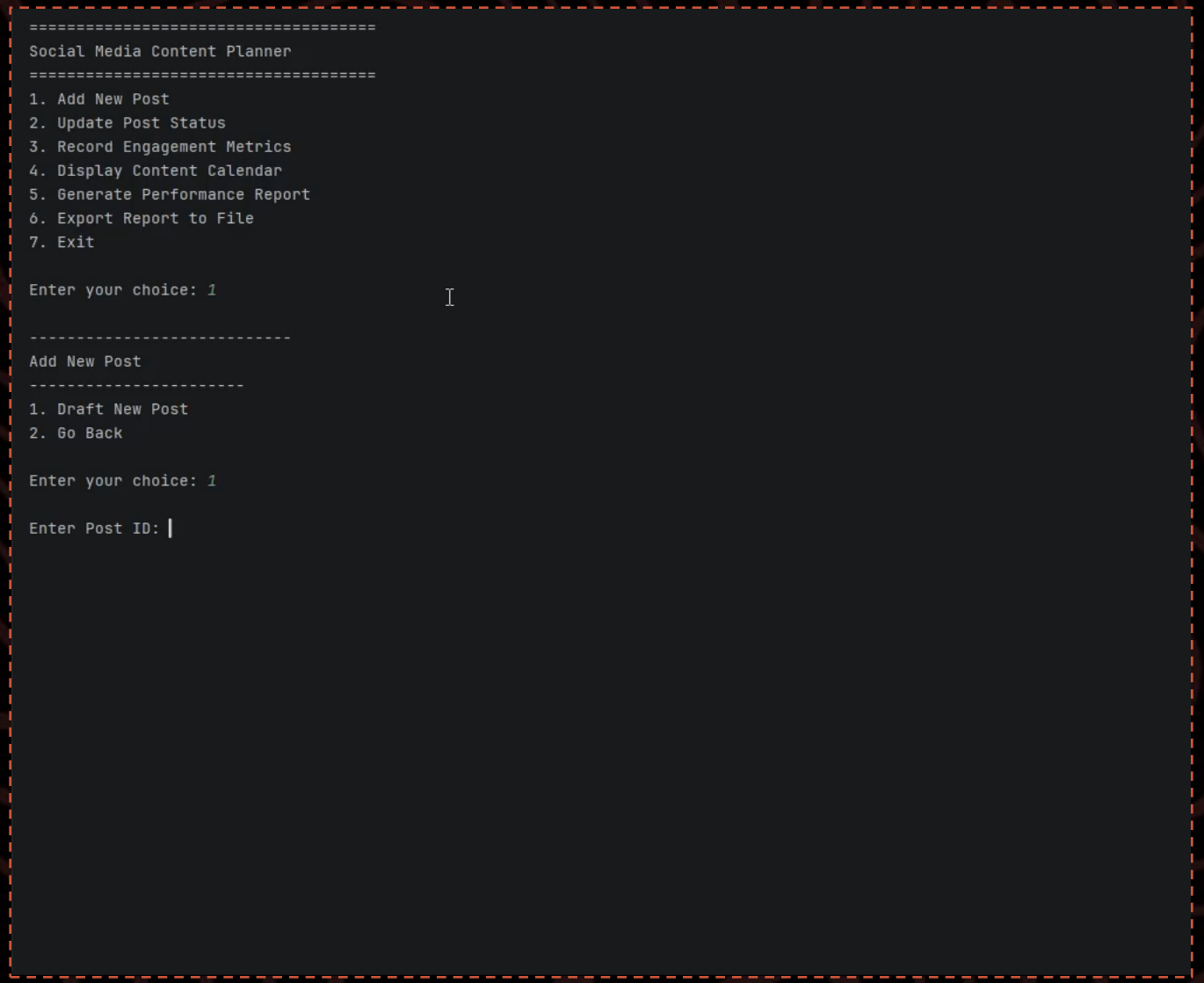
FILE NON-EXISTENCE PREVENTION

```
def file_check(file_name):  #checks whether core files exists
    try:
        with open(f"{file_name}", "r") as file:   #try to open the file in read mode,
            pass
    except FileNotFoundError:   #if an error occurred, that means the file does not exist
        print(f"{file_name} does not exists, creating a new file.")
        if file_name == "platforms.txt":     #adds valid platforms to new file
            with open(f"{file_name}", "a") as file:
                file.write("1,Facebook\n"
                           "2,Tiktok\n"
                           "3,Instagram\n"
                           "4,X")
            print("All valid platforms automatically filled in")
        else:
            with open(f"{file_name}", "w") as file:
                pass
        pass
```

SHOWCASE

INPUT VALIDATIONS

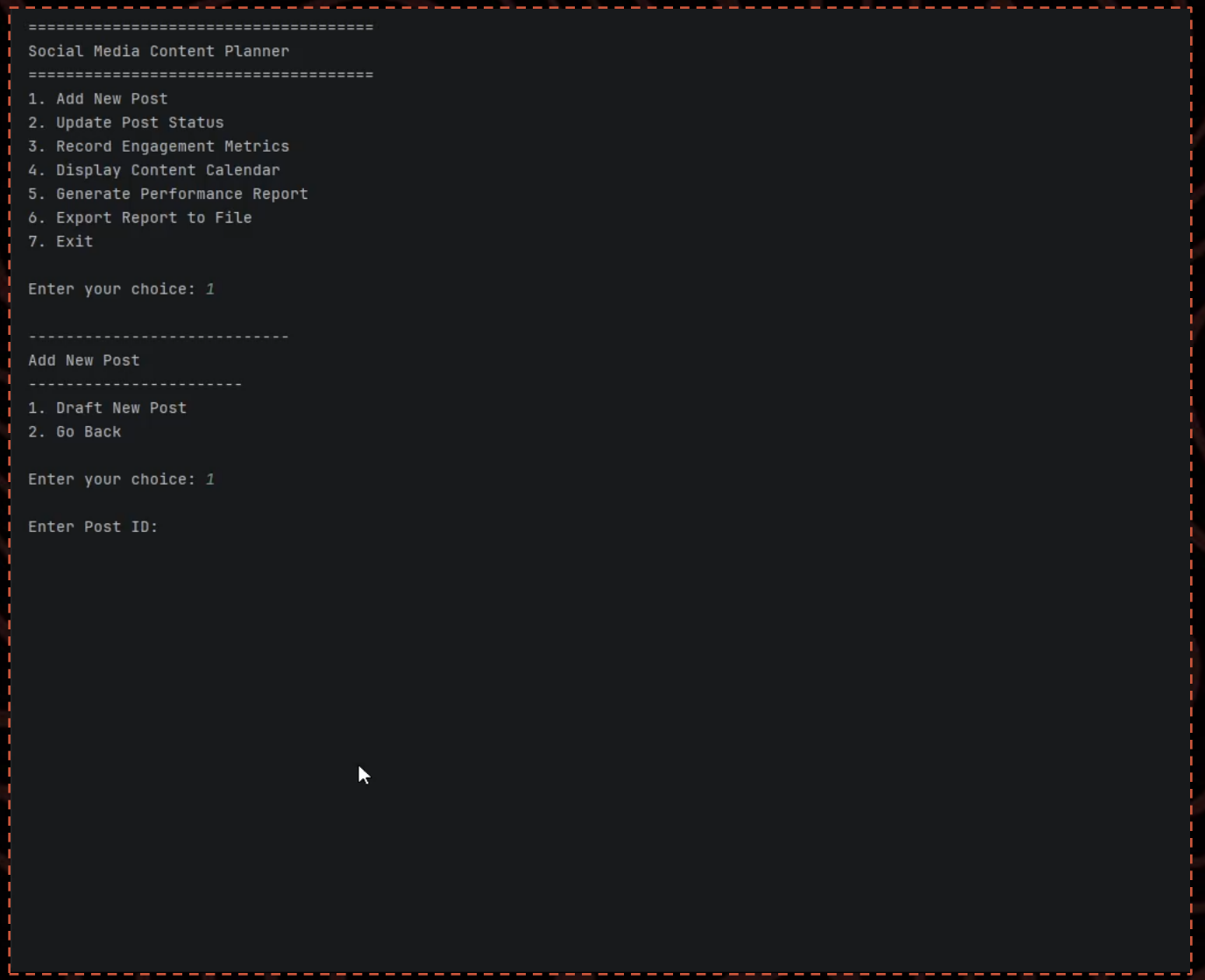
EMPTY INPUT



SHOWCASE

INPUT VALIDATIONS

NEGATIVE INTEGER



SHOWCASE

INPUT VALIDATIONS

INVALID PLATFORM

```
==================================
Social Media Content Planner
==================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 1

--------------------------
Add New Post
----------------------
1. Draft New Post
2. Go Back

Enter your choice: 1

Enter Post ID: 101

Available Platforms
- Facebook
- Tiktok
- Instagram
- X

Enter Platform:
```

SHOWCASE

INPUT VALIDATIONS

DUPLICATE POST ID

```
==================================
Social Media Content Planner
==================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 1

--------------------------
Add New Post
----------------------
1. Draft New Post
2. Go Back

Enter your choice: 1

Enter Post ID:
```

SHOWCASE

INPUT VALIDATIONS

INVALID DATE FORMAT

```
==================================
Social Media Content Planner
==================================
1. Add New Post
2. Update Post Status
3. Record Engagement Metrics
4. Display Content Calendar
5. Generate Performance Report
6. Export Report to File
7. Exit

Enter your choice: 1

--------------------------
Add New Post
----------------------
1. Draft New Post
2. Go Back

Enter your choice: 1

Enter Post ID: 101

Available Platforms
- Facebook
- Tiktok
- Instagram
- X

Enter Platform: tiktok
Enter Caption: This is the best program ever!
Enter Scheduled Date (DD/MM/YYYY):
```

SHOWCASE

INPUT VALIDATIONS

DUPLICATE ENGAGEMENT ENTRY

```
Record Engagement Metrics
----------------------
Please choose from one of these posts to log engagement data.

['7', 'Facebook', 'Customer testimonial', '02/09/2026']
['12', 'Facebook', 'Giveaway alert', '07/01/2026']
['16', 'Instagram', 'Company update', '24/03/2026']
['21', 'Instagram', 'Customer testimonial', '10/12/2026']
['24', 'Facebook', 'Company update', '06/01/2026']
['28', 'Instagram', 'Holiday special', '30/01/2026']
['31', 'Facebook', 'Industry news', '13/10/2026']
['35', 'X', 'Community spotlight', '13/09/2026']
['36', 'Instagram', 'Giveaway alert', '19/11/2026']
['38', 'TikTok', 'New product launch!', '20/12/2026']
['40', 'Facebook', 'Did you know?', '21/03/2026']
['42', 'TikTok', 'Weekly recap', '21/04/2026']
['43', 'Facebook', 'Community spotlight', '04/05/2026']
['44', 'TikTok', 'Flash sale', '14/04/2026']
['45', 'TikTok', 'Customer testimonial', '31/03/2026']
['49', 'Instagram', 'Did you know?', '09/08/2026']
['50', 'TikTok', 'New product launch!', '04/10/2026']
['53', 'TikTok', 'Did you know?', '20/09/2026']
['55', 'Facebook', 'Holiday special', '28/10/2026']
['56', 'X', 'Q&A session', '22/07/2026']
['57', 'Facebook', 'Holiday special', '26/08/2026']
['58', 'Instagram', 'Giveaway alert', '29/05/2026']
['62', 'TikTok', 'New product launch!', '04/06/2026']
['68', 'Instagram', 'Customer testimonial', '25/11/2026']
['69', 'Instagram', 'Behind the scenes', '13/07/2026']
['76', 'TikTok', 'New product launch!', '26/02/2026']
['77', 'TikTok', 'Company update', '04/09/2026']
['80', 'Facebook', 'Flash sale', '19/03/2026']
['82', 'TikTok', 'Giveaway alert', '24/09/2026']
['85', 'TikTok', 'Flash sale', '24/02/2026']
['87', 'Facebook', 'Company update', '24/11/2026']
['88', 'TikTok', 'Giveaway alert', '13/04/2026']
['89', 'Facebook', 'Did you know?', '11/04/2026']
['94', 'Instagram', 'Giveaway alert', '11/11/2026']
['95', 'TikTok', 'Weekly recap', '03/05/2026']
['98', 'X', 'Customer testimonial', '23/08/2026']
======================================================================
Enter your post ID [Q to quit]:
```

EXPLANATION

EMPTY INPUT

```
def validate_required_input(prompt):   # validates against empty fields
    while True:
        value = input(prompt).strip()
        if value != "":
            return value
        print("--> Input cannot be empty. Please try again.")
```

NEGATIVE INTEGER

```
def validate_positive_integer(prompt):   # makes sure all inputs for record engagement metrics are positive
    while True:
        try:
            number = int(input(prompt))
            if number >= 0:
                return number
            else:
                print("--> Number cannot be negative.")
        except ValueError:
            print("--> Enter only a positive integer.")
```

INVALID PLATFORM

```
def validate_platform(platform_input):   # runs check if user input of platofrm exists, if yes True, otheriwse its False
    try:
        with open("platforms.txt", "r") as file:
            for line in file:
                data = line.strip().split(",")
                # platform name is stored at index 1
                if data[1].lower() == platform_input.lower():
                    return True
    except FileNotFoundError:
        print("--> platform not found.")
    return False
```

DUPLICATE POST ID

```
def validate_unique_id(post_id):    # Checks if POSTID is unique and numeric
    if not post_id.isdigit():
        return print("--> Only positive integer ID's are accepted")

    try:
        with open("posts.txt", "r") as file:
            for line in file:
                data = line.strip().split(",")
                if data[0].upper() == post_id.upper():
                    return print("--> The ID you entered isn't unique, enter unique ID.")
    except FileNotFoundError:
        # File does not exist yet, therefore no duplicates
        pass
    return True
```

INVALID DATE FORMAT

```
def validate_date(prompt):   # efnorces proper date fromat
    while True:
        date_input = input(prompt).strip()
        try:
            datetime.datetime.strptime(date_input, "%d/%m/%Y")
            return date_input
        except ValueError:
            print("-> Invalid date. Please enter date in DD/MM/YYYY format.")
```

DUPLICATE ENGAGEMENT ENTRY

```
def id_verification(userin):
    #checks whether user input is inside engagement.txt
    with open("engagement.txt", "r") as file:
        for row in file:
            data = row.strip().split(",")
            if data[0] == userin.upper():
                return True
    return False
    #remvoed int_validity and replaced with validate_positive_integer - Abbas
```

THANK YOU